



Beyond Trust: Building Community-Driven Security Analysis for Your .NET Software Supply Chain

Niels Tanis

Security Researcher & Engineer

TIDALIS



Who am I?

#UCK26

- Niels Tanis
- Security Researcher & Engineer
 - Background .NET Development, Pentesting/ethical hacking, and software security consultancy
 - Decade of research on static analysis for .NET apps
 - Principal Software Security Engineer at Tidalis
- Microsoft MVP – Developer Technologies



TIDALIS



@niels.fennec.dev

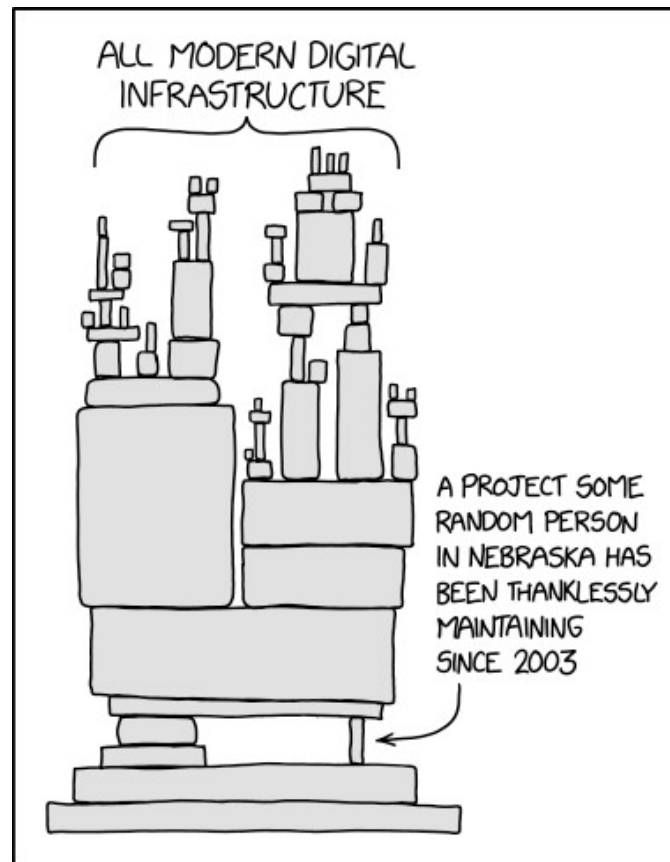


@nielstanis@infosec.exchange

Modern Application Architecture

XKCD 2347

#UCK26



TIDALIS



@niels.fennec.dev

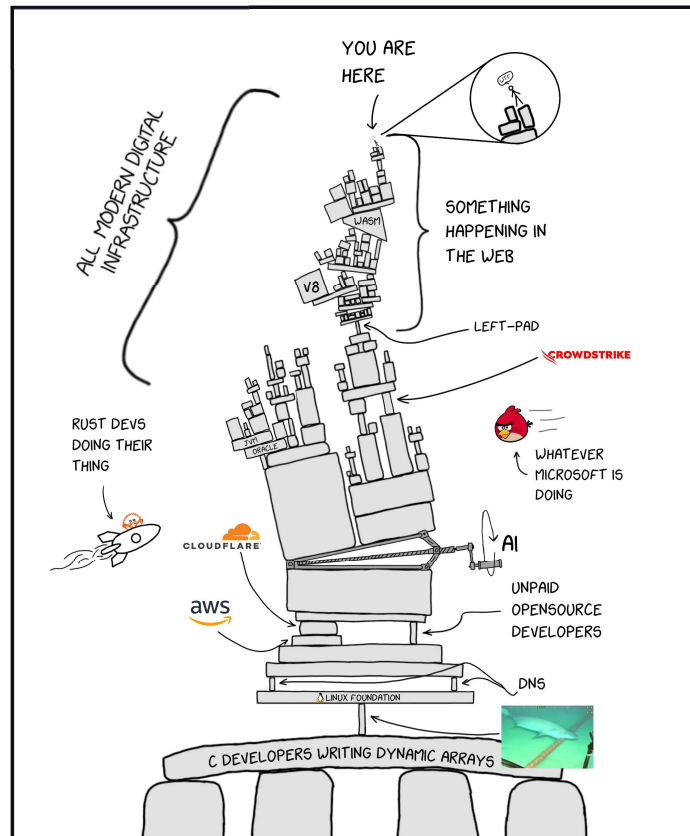


@nielstanis@infosec.exchange

You are here



#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

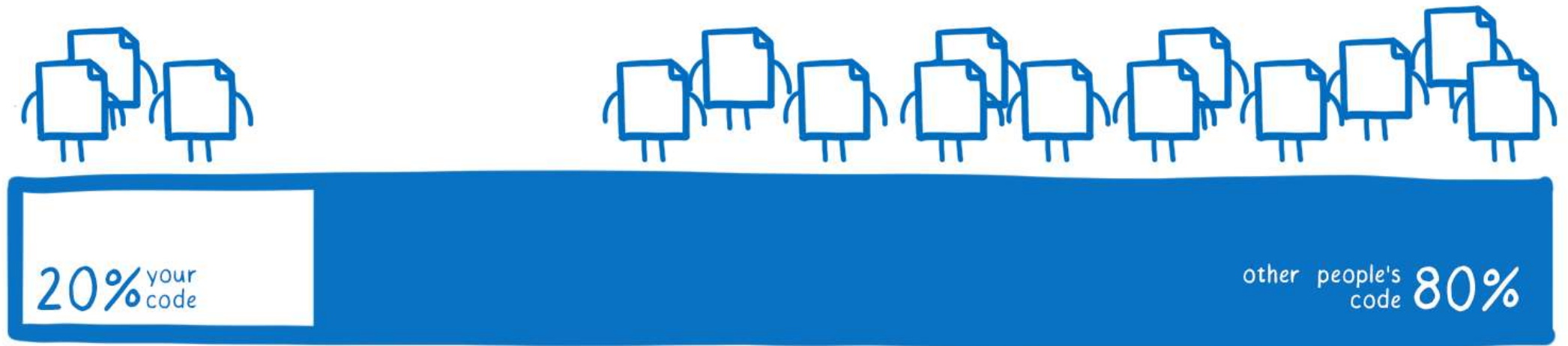
Agenda

#UCK26

- Security challenges in our .NET software supply chain
- How can we start securing our supply-chain?
- Fennec Labs
 - Understanding what projects do for security
 - Know what's inside package
 - Review package
 - Future and idea's
- Conclusion and Q&A

Average codebase composition

#UCK26



TIDALIS



@niels.fennec.dev



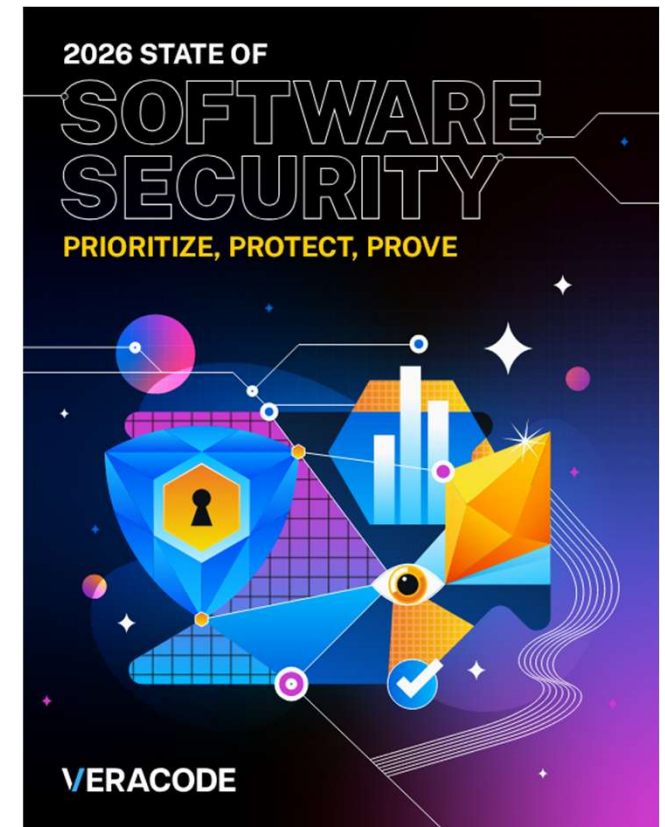
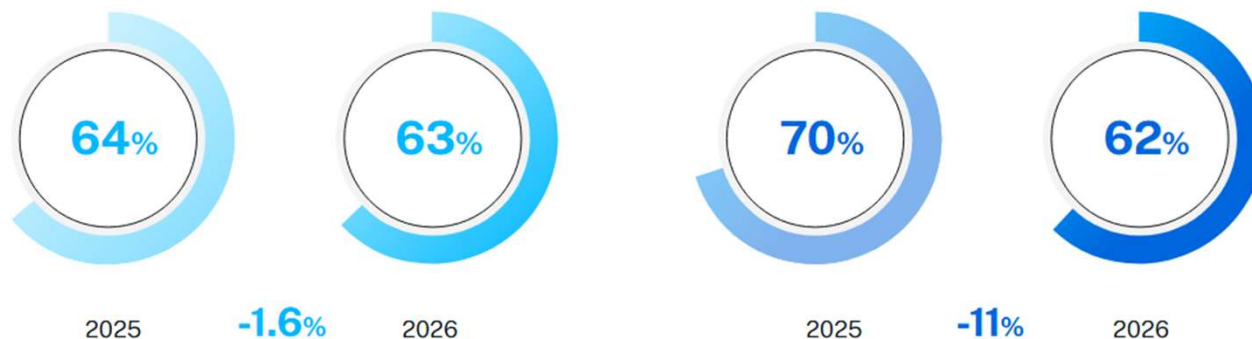
@nielstanis@infosec.exchange

Veracode State Of Software Security 2026

#UCK26

Prevalence of flaws
in first-party (left)
vs. third-party
(right) code among
applications

Percentage of flaws



TIDALIS



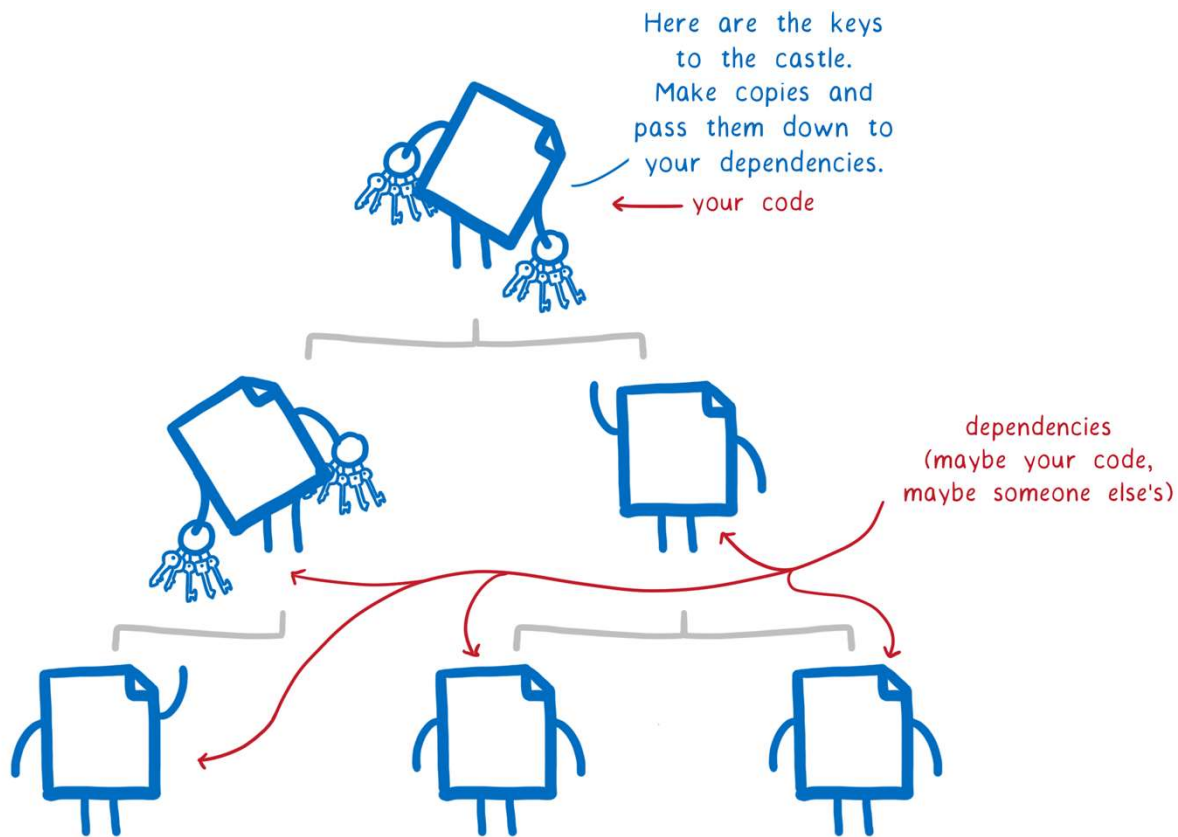
@niels.fennec.dev



@nielstanis@infosec.exchange

Average codebase composition

#UCK26



TIDALIS

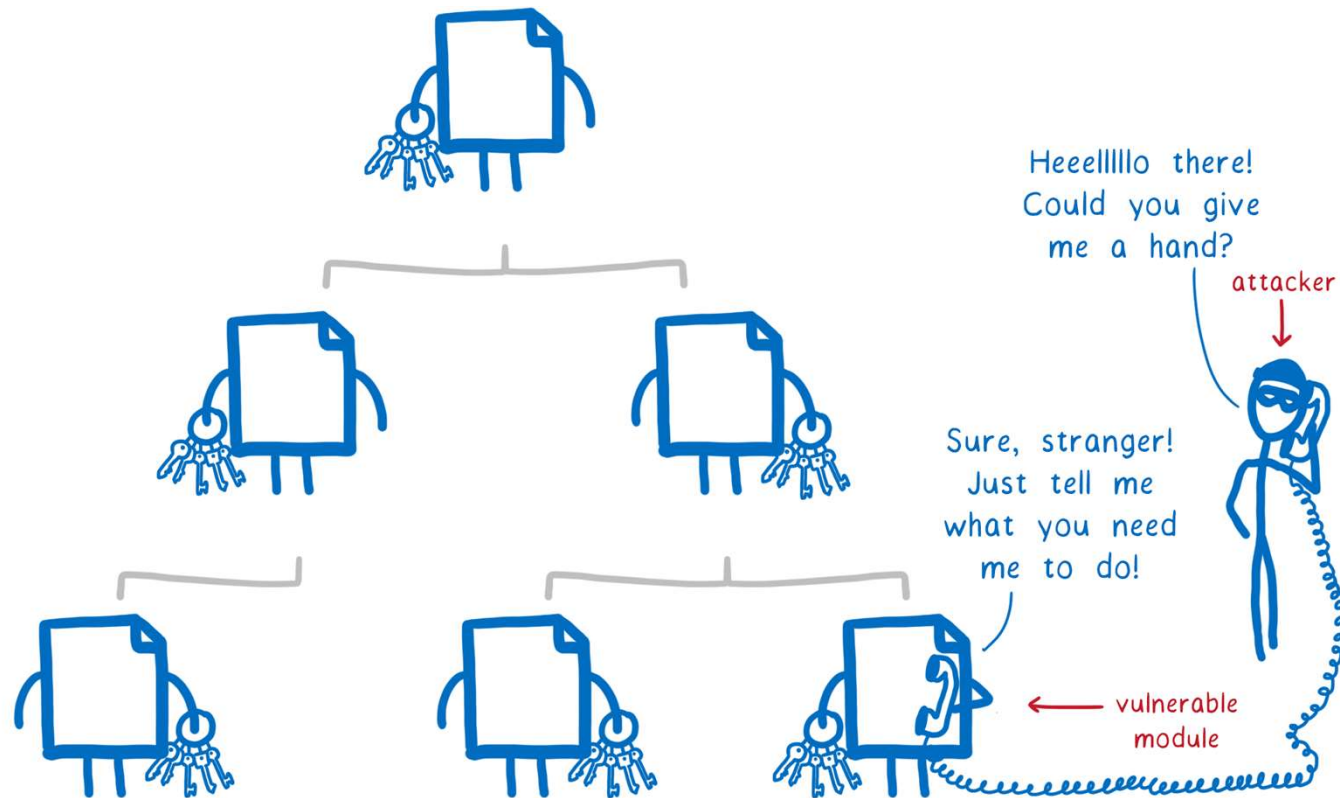
@niels.fennec.dev

၉

@nielstanis@infosec.exchange

Vulnerable Assembly

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
└─┬─ nelson at ghost-m2 in ~/research/consoleapp
   │ dotnet list package
   └─ Project 'consoleapp' has the following package references
      [net9.0]:
      Top-level Package      Requested  Resolved
      > itext7                9.4.0     9.4.0

└─┬─ nelson at ghost-m2 in ~/research/consoleapp
   │ dotnet list package --vulnerable
   └─ The following sources were used:
      https://api.nuget.org/v3/index.json

      The given project `consoleapp` has no vulnerable packages given the current sources.
└─┬─ nelson at ghost-m2 in ~/research/consoleapp
   │
   └─
```



DotNet CLI



```
nelson@ghost-m2: ~/research/consoleapp
└─ nelson at ghost-m2 in ~/research/consoleapp
   └─ dotnet list package --include-transitive
      Project 'consoleapp' has the following package references
      [net9.0]:
      Top-level Package      Requested  Resolved
      > itext7                9.4.0     9.4.0

      Transitive Package                                Resolved
      > itext                9.4.0
      > itext.common          9.4.0
      > Microsoft.CSharp      4.0.1
      > Microsoft.DotNet.PlatformAbstractions 1.1.0
      > Microsoft.Extensions.DependencyInjection 5.0.0
      > Microsoft.Extensions.DependencyInjection.Abstractions 5.0.0
      > Microsoft.Extensions.DependencyModel 1.1.0
      > Microsoft.Extensions.Logging 5.0.0
      > Microsoft.Extensions.Logging.Abstractions 5.0.0
      > Microsoft.Extensions.Options 5.0.0
      > Microsoft.Extensions.Primitives 5.0.0
      > Microsoft.NETCore.Platforms 1.1.1
      > Microsoft.NETCore.Targets 1.1.3
      > Microsoft.Win32.Primitives 4.3.0
      > Microsoft.Win32.Registry 4.3.0
      > Newtonsoft.Json        9.0.1
```



DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
> dotnet list package --include-transitive --vulnerable

The following sources were used:
  https://api.nuget.org/v3/index.json

Project `consoleapp` has the following vulnerable packages
[net9.0]:
  Transitive Package      Resolved  Severity  Advisory URL
  > Newtonsoft.Json       9.0.1     High      https://github.com/advisories/GHSA-5crp-9r3c-p9vr

nelson at ghost-m2 in ~/research/consoleapp
>
```



DotNet CLI - .NET 10



```
nelson@ghost-m2:~/research/consoleapp
> nelson at ghost-m2 in ~/research/consoleapp
└─ dotnet --version
10.0.100
> nelson at ghost-m2 in ~/research/consoleapp
└─ dotnet build
Restore succeeded with 1 warning(s) in 0.3s
  /Users/nelson/Research/consoleapp/consoleapp.csproj : warning NU1903: Package 'Newtonsoft.Json' 9.0.1 has a known high severity vulnerability, https://github.com/advisories/GHSA-5crp-9r3c-p9vr
  consoleapp net10.0 succeeded with 1 warning(s) (1.5s) → bin/Debug/net10.0/consoleapp.dll
  /Users/nelson/Research/consoleapp/consoleapp.csproj : warning NU1903: Package 'Newtonsoft.Json' 9.0.1 has a known high severity vulnerability, https://github.com/advisories/GHSA-5crp-9r3c-p9vr

Build succeeded with 2 warning(s) in 2.0s
> nelson at ghost-m2 in ~/research/consoleapp
└─
```



NuGet Package Pruning



A screenshot of a web browser displaying a Microsoft Dev Blogs article. The browser's address bar shows the URL: devblogs.microsoft.com/dotnet/nuget-package-pruning-in-dotnet-10/?hide_banner=true. The page header includes the Microsoft logo, 'Dev Blogs', and navigation links for AI, .NET, Developer, Platform and Tools, Data, and Windows. The article is dated May 18th, 2026, and has 2 reactions. The author is Nikolche Kolev, a Principal Software Engineer. The article text discusses NuGet Audit warnings for transitive packages like System.Text.Json and System.Text.Encodings.Web, and mentions that in .NET 10, NuGet audits transitive dependencies by default, which can reduce vulnerability reports by 70% compared to previous defaults. A table of contents on the right lists sections: Background: How We Got Here, What is Package Pruning?, Before and After, What Changed in .NET 10, and Summary. On the left side of the article, there are three icons with counts: a speech bubble with '2', a comment bubble with '3', and a share icon with '5'.

TIDALIS



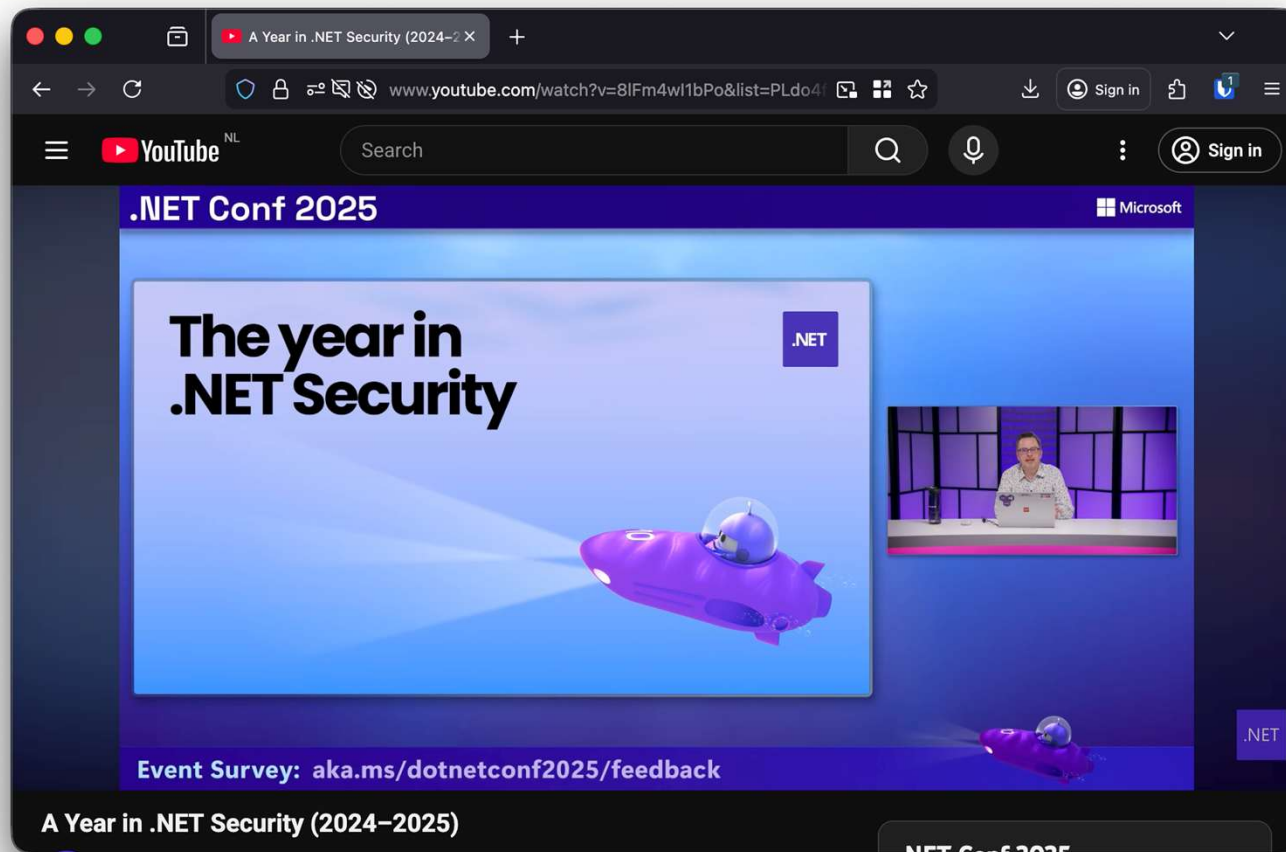
@niels.fennec.dev



@nielstanis@infosec.exchange

.NET Conf 2025 – The year in .NET Security

#UCK26

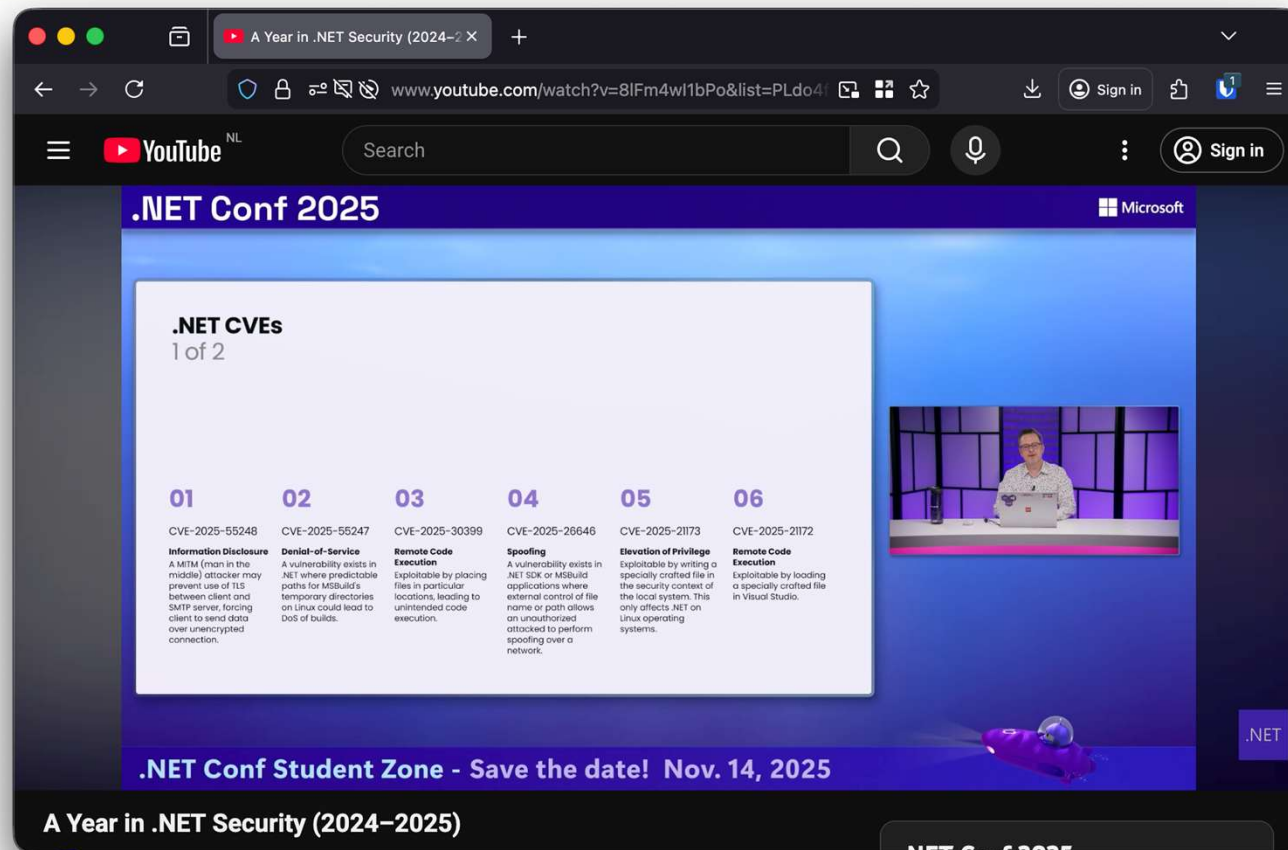


TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

.NET Conf 2025 – The year in .NET Security

#UCK26



TIDALIS



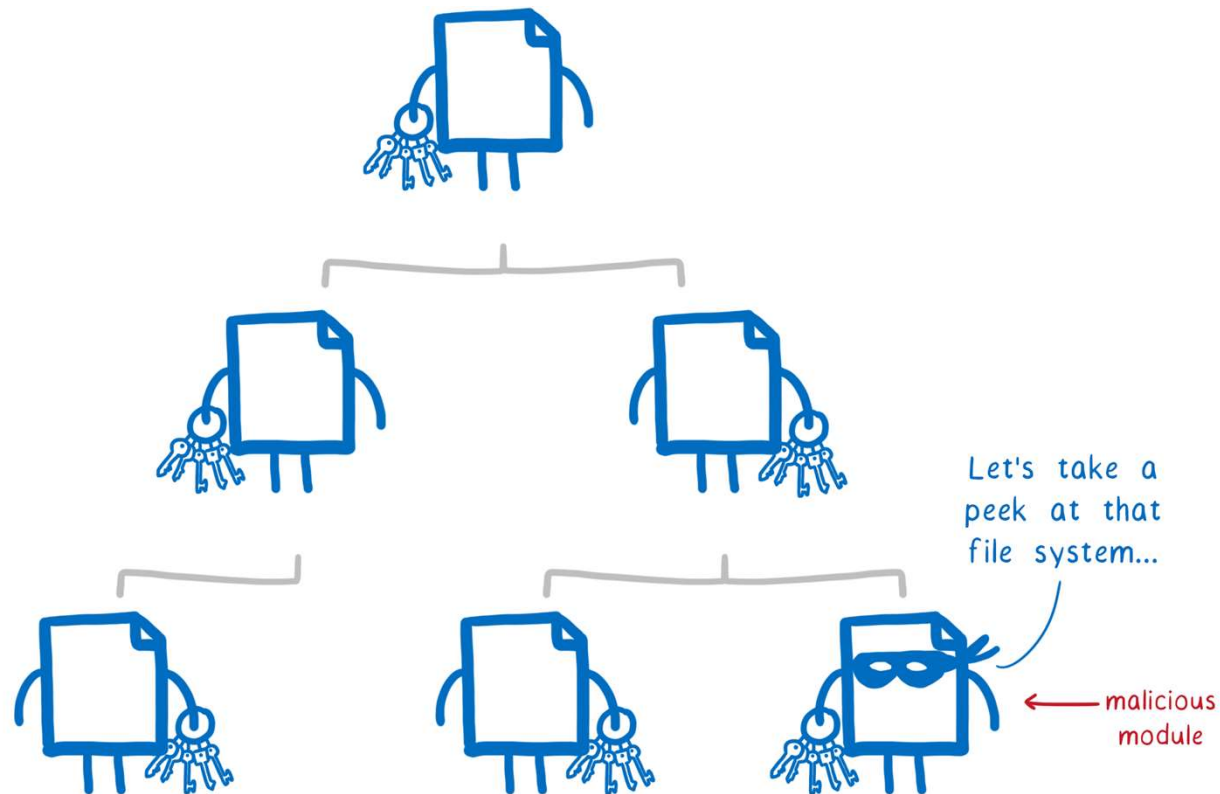
@niels.fennec.dev



@nielstanis@infosec.exchange

Malicious Assembly

#UCK26



TIDALIS



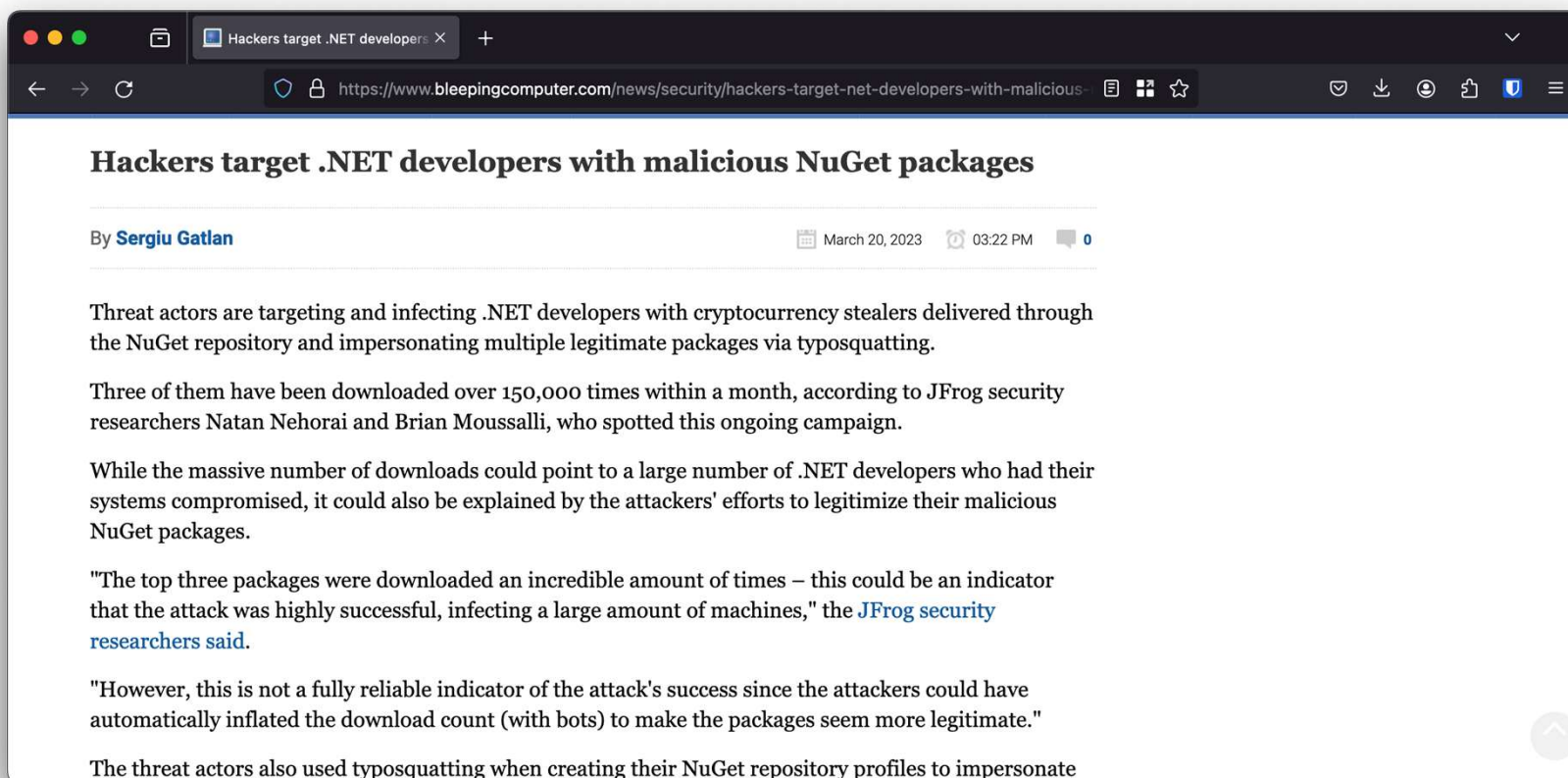
@niels.fennec.dev



@nielstanis@infosec.exchange

Malicious Package

#UCK26



TIDALIS



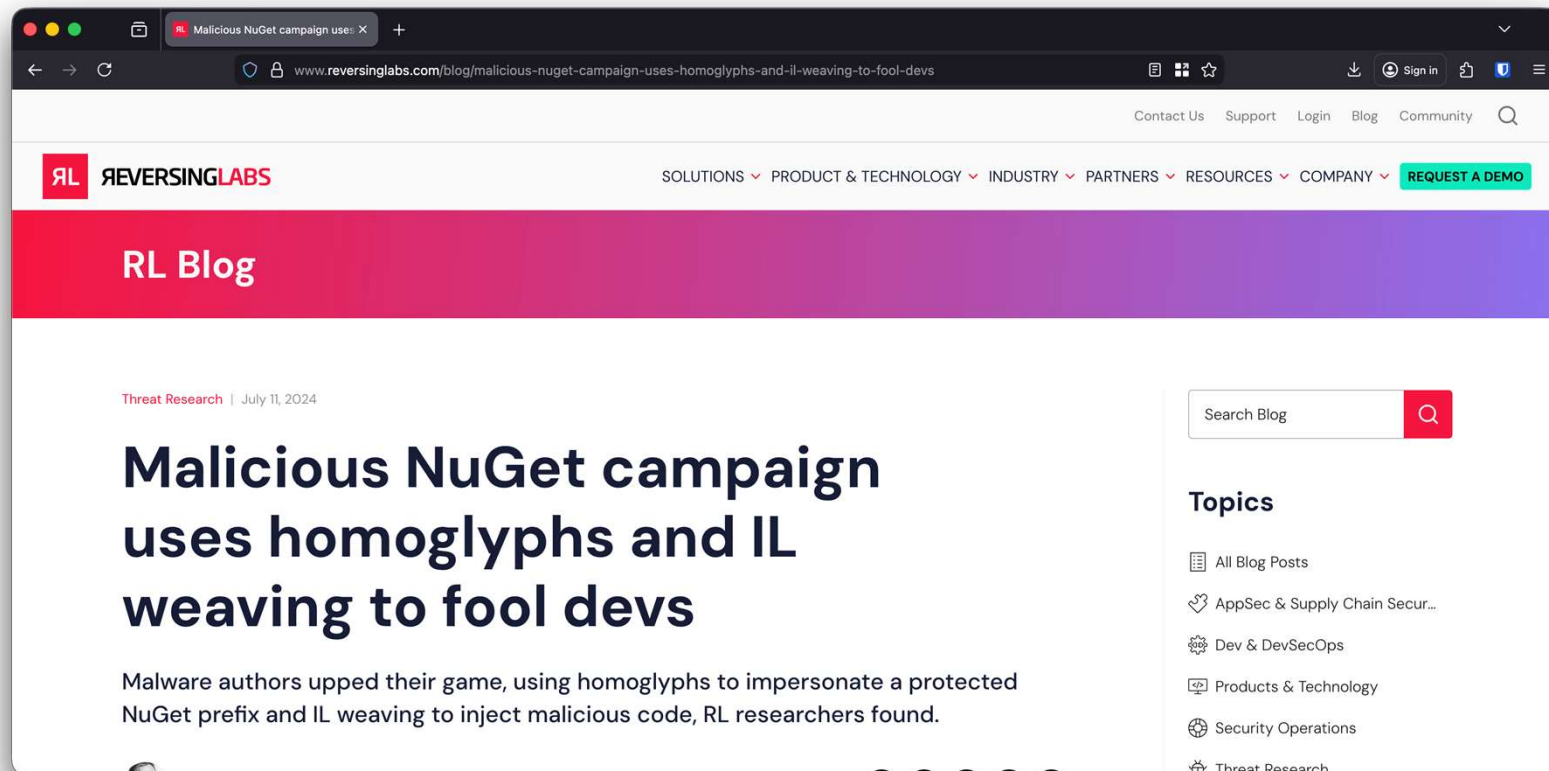
@niels.fennec.dev



@nielstanis@infosec.exchange

Malicious Package

#UCK26



TIDALIS



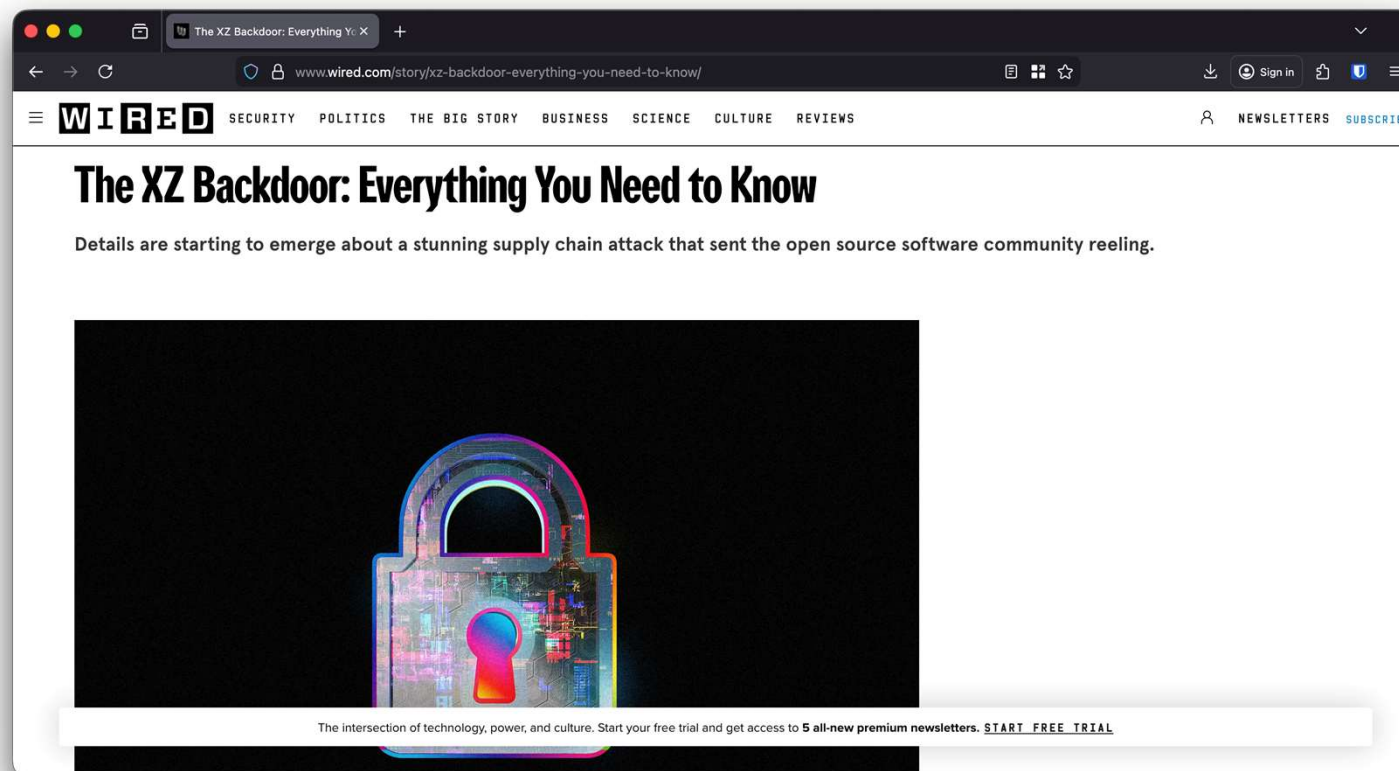
@niels.fennec.dev



@nielstanis@infosec.exchange

Do you know what's inside? – xz utils

#UCK26



TIDALIS



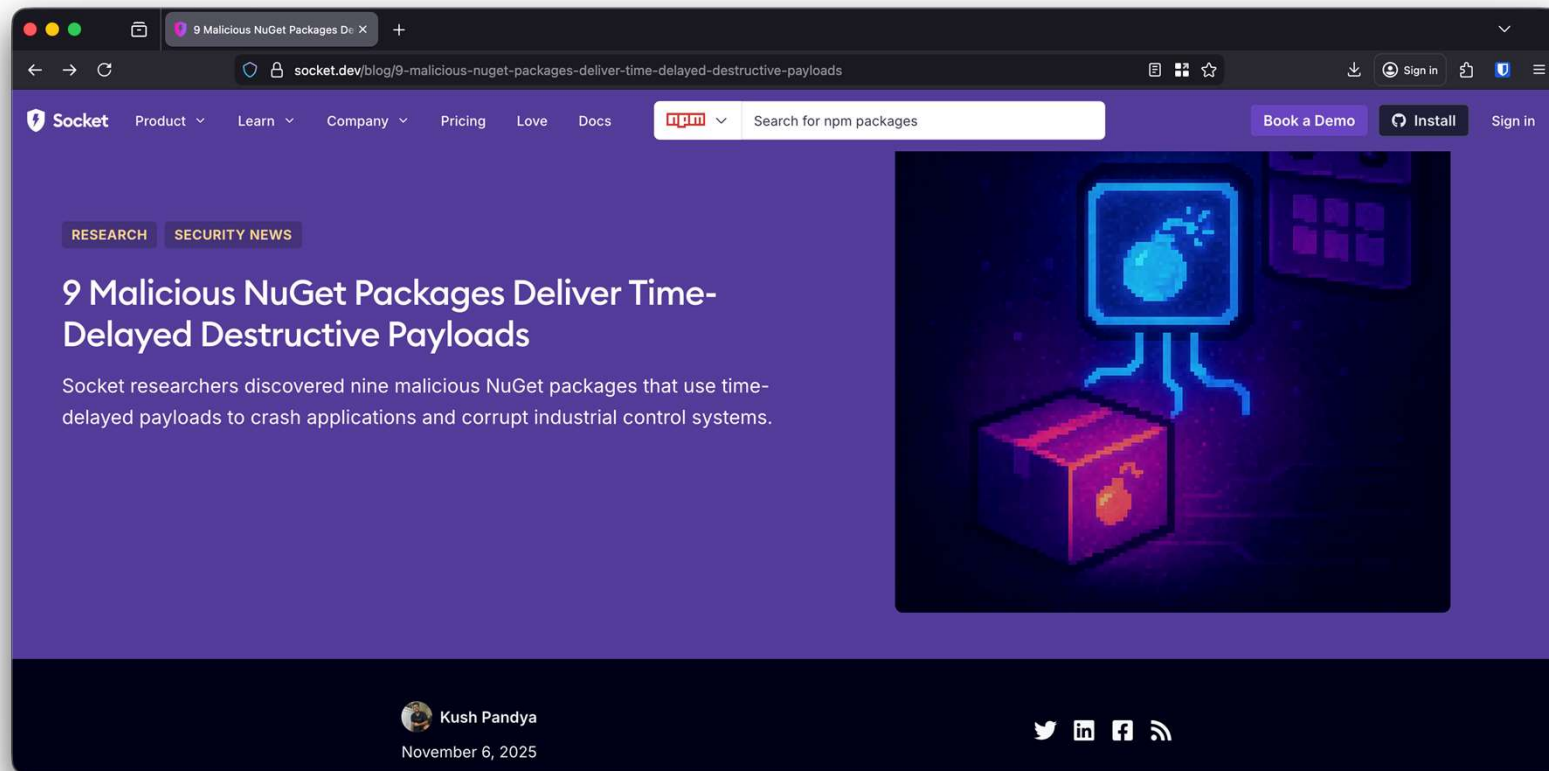
@niels.fennec.dev



@nielstanis@infosec.exchange

Malicious Package

#UCK26



TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

Malicious Package

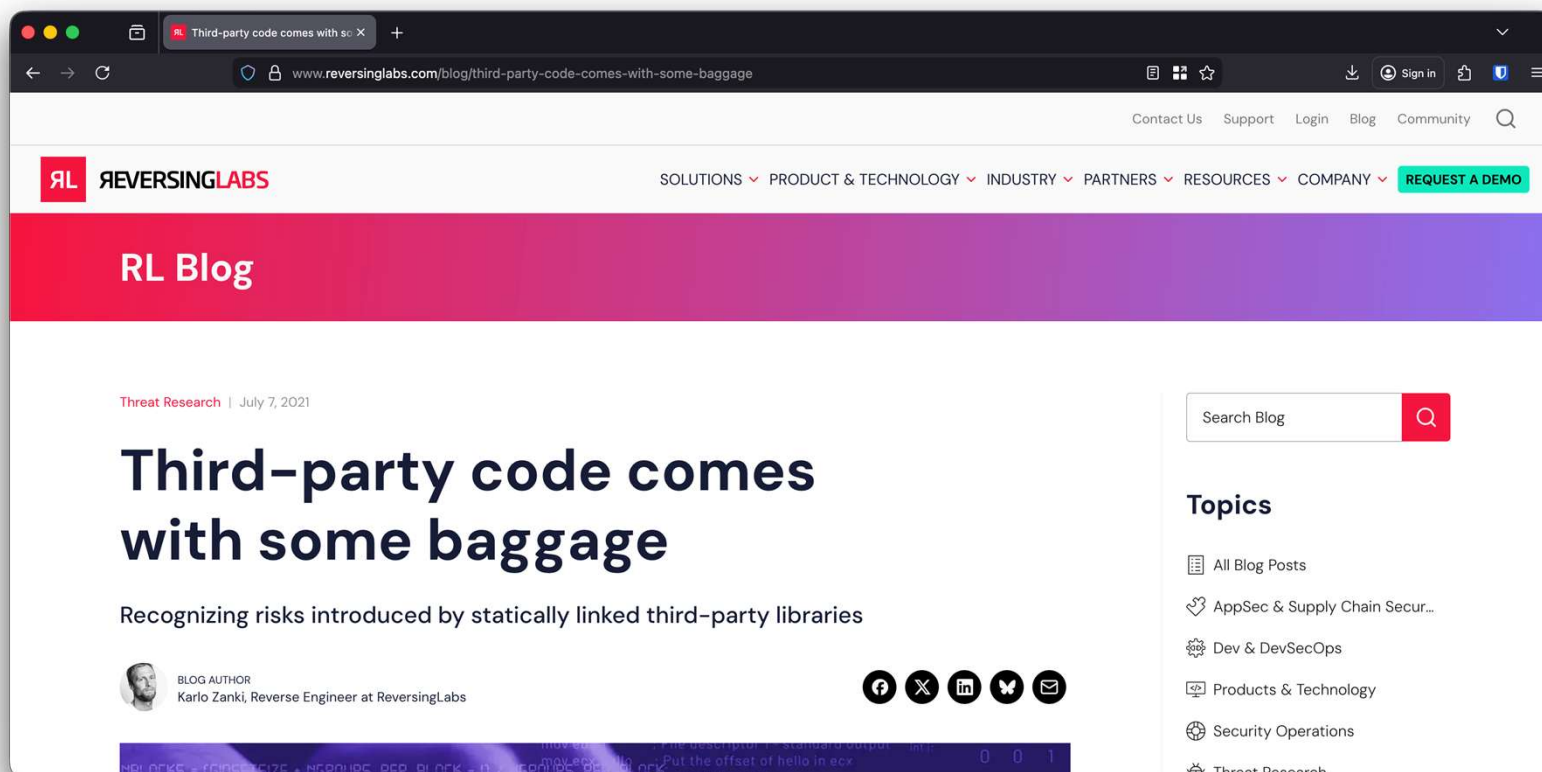


- Single publisher called **shanghai666**
- 99% code is valid & functional focusing on databases and industrial PLC's, malicious extension methods
- Delayed logic for Aug '27 & Nov '28
- 20% of database queries are terminated
- Sharp7Extend (typo squatting) for PLC, immediate activation mimics hardware failures



Do you know what's inside?

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

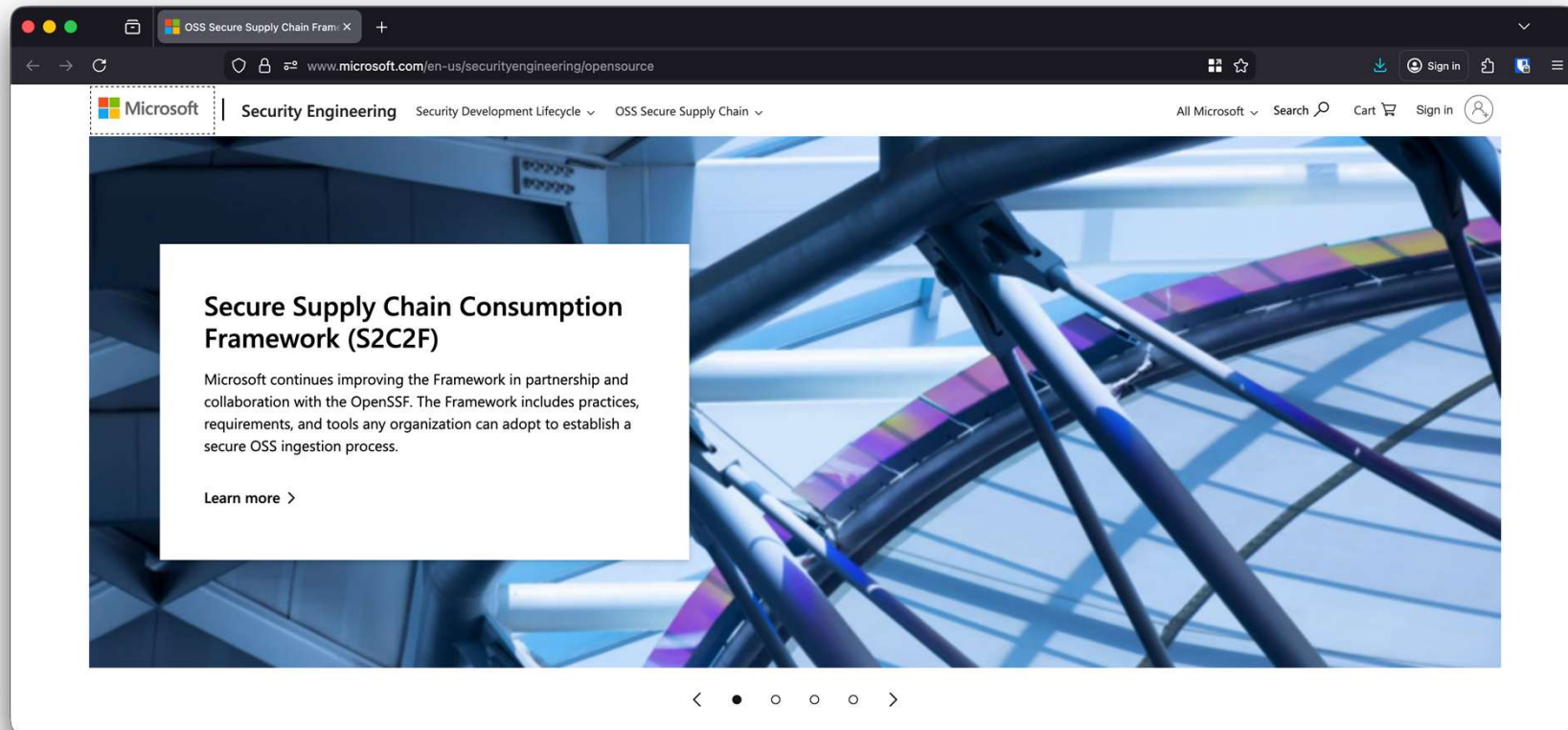
Do you know what's inside?

#UCK26

- Unmanaged components interop
- Precompiled version of 7Zip, WinSCP and PuTTYgen
- Zlib compression library
 - Only source code distributed
 - Used in medical appliances
 - Severe vulnerabilities since 2005!

Secure Supply Chain Consumption Framework (S2C2F)

#UCK26



TIDALIS



@niels.fennec.dev



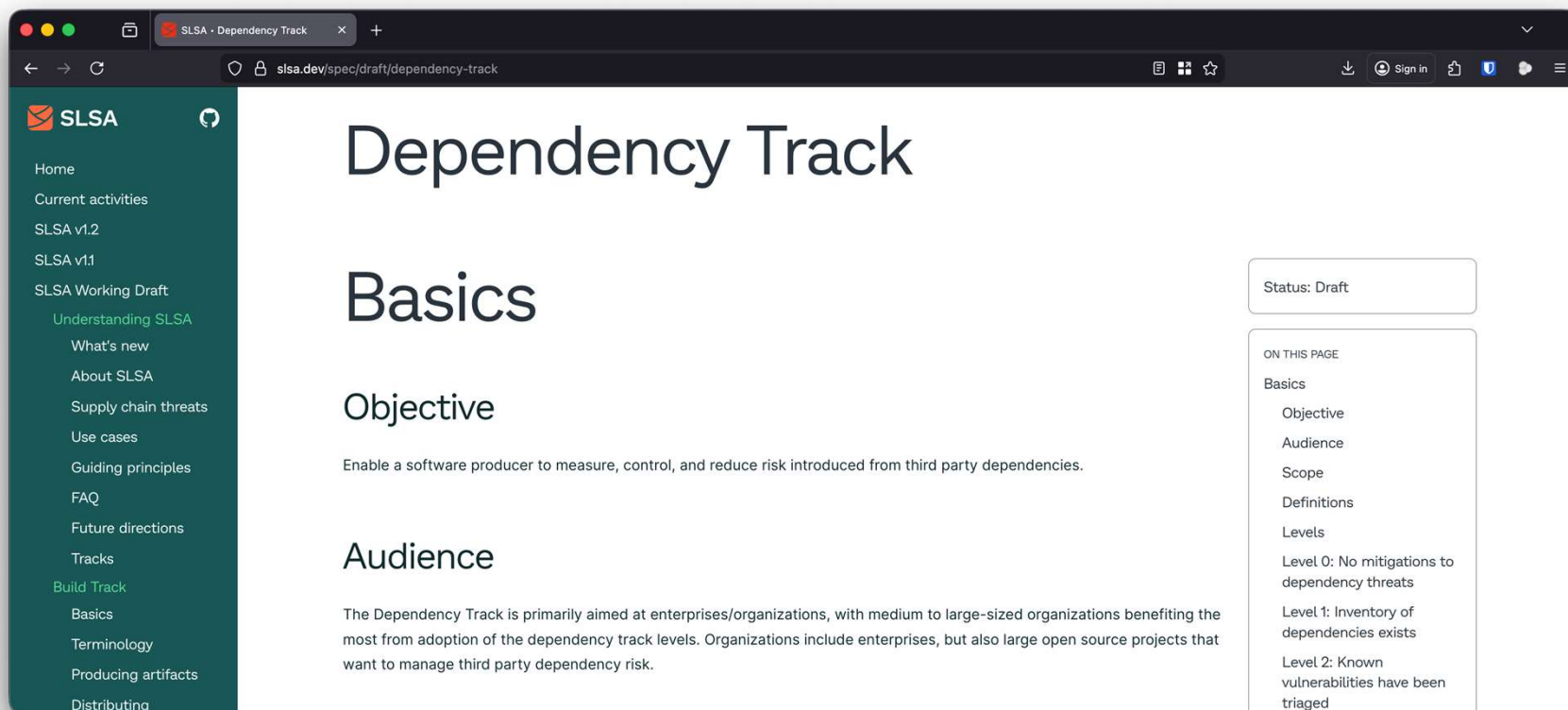
@nielstanis@infosec.exchange

Secure Supply Chain Consumption Framework (S2C2F)



Level 1	Level 2	Level 3	Level 4
 Minimum OSS Governance Program • Use package managers [ING-1] • Local copy of artifact [ING-2] • Scan with known vulns [SCA-1] • Scan for software licenses [SCA-2] • Inventory OSS [INV-1] • Manual OSS updates [UPD-1]	 Secure Consumption and Improved MTTR • Scan for end of life [SCA-3] • Have an incident response plan [INV-2] • Auto OSS updates [UPD-2] • Alerts on vulns at PR time [UPD-3] • Audit that consumption is through approved ingestion method [AUD-2] • Validate integrity of OSS [AUD-3] • Secure package source file configuration [ENF-1]	 Malware Defense and Zero-Day Detection • Deny list capability [ING-3] • Clone OSS source [ING-4] • Scan for malware [SCA-4] • Proactive security reviews [SCA-5] • Enforce OSS provenance [AUD-1] • Enforce consumption from curated feed [ENF-2]	 Advanced Threat Defense • Validate the SBOMs of OSS consumed [AUD-4] • Rebuild OSS on trusted infrastructure [REB-1] • Digitally sign rebuilt OSS [REB-2] • Generate SBOM for rebuilt OSS [REB-3] • Digitally sign protected SBOMs [REB-4] • Implement fixes [FIX-1]

S2C2F → SLSA Dependency Track



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Hot take; dependency cooldowns



What's inside? (Audit)

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Open SSF Security Scorecard

#UCK26

- Set Automated Security Checks
- Scoring System 0-10
- Security Best Practices
- Ease of Use
- Community Support



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Maintenance



- Vulnerabilities (**High**)
 - Does the project have unfixed vulnerabilities?
- Maintained (**High**)
 - Is the project at least 90 days old, and maintained?
- Security Policy (**Medium**)
 - Does project contain security policy?

- License (**Low**)
 - Does the project declare a licence?
- CII Best Practices (**Low**)
 - Does the project have a CII Best Practices Badge?
- Dependency Update Tool (**High**)
 - Does the project use tools to help update its dependencies e.g. Dependabot, RenovateBot?

Continuous Testing



- Continuous Integration Tests (**Low**)
 - Does the project run tests in CI?
- Fuzzing (**Medium**)
 - Does the project use fuzzing tools, e.g. OSS-Fuzz?
- Static Analysis (**Medium**)
 - Does the project use static code analysis tools?

Source Risk Assessment



- Binary Artifacts (**High**)
 - Does repository contain binaries/compiled artifacts?
- Branch Protection (**High**)
 - GitHub best practices branch protection
- Dangerous Workflow (**Critical**)
 - GitHub best practices for Workflow Actions

Source Risk Assessment



- Code Review (**High**)
 - Does the project require code review before code is merged?
- Contributors (**Low**)
 - Does the project have contributors from multiple organizations?

Build Risk Assessment



- Pinned Dependencies (**Medium**)
 - Does the project declare and pin dependencies?
- Token Permission (**High**)
 - Does the project declare GitHub workflow tokens as read only?

Build Risk Assessment



- Packaging (**Medium**)
 - Does the project build and publish official packages from CI/CD?
- Signed Releases (**High**)
 - Does the project cryptographically sign releases?

Demo OpenSSF Scorecard



Running checks

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Scorecard

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Instrument

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Application Inspector

#UCK26

The screenshot shows the Application Inspector web interface. The browser address bar displays the file path: `file:///C:/Demo/Appinspector/publish/output.html#`. The page title is "Application Inspector" with navigation links for Overview, Summary, Features, and About. The main heading is "Application Features". Below this, a paragraph explains that the section reports major characteristics of the application or its primary features organized by customizable Feature Groups. It instructs users to click on highlighted icons to view details or expand a feature group for more information. To view where in source code a specific feature was found, users should click the Rule name link shown on the right. A disabled icon indicates a not found status for that feature.

Feature Groups

Feature Group	Icons
+ Select Features	Icons for user, lock, key, monitor, folder, and eye
+ General Features	Icons for folder, document, trash, link, file, folder, folder, and folder
+ Development	Icons for folder, folder, folder, folder, folder, and folder
+ Active Content	Icons for folder, folder, folder, folder, folder, and folder
+ Data Storage	Icons for folder, folder, folder, folder, folder, and folder
+ Sensitive Data	Icons for folder, folder, folder, folder, folder, and folder
+ Cloud Services	Icons for folder, folder, folder, folder, folder, and folder
+ OS Integration	Icons for folder, folder, folder, folder, folder, and folder
+ OS System Changes	Icons for folder, folder, folder, folder, folder, and folder
+ Other	Icons for folder, folder, folder, folder, folder, and folder

Associated Rules

Name (click to view source)
Authentication: Microsoft (Identity)
Authentication: General
Authentication: (OAuth)

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Application Inspector

#UCK26

Select Features

Feature	Confidence	Details
 Authentication		View
 Authorization		View
 Cryptography		View
 Object Deserialization		N/A
 AV Media Parsing		N/A
 Dynamic Command Execution		N/A

TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

.NET Reproducibility



- Reproducible builds
 - Independent path from source to binary code.
- Roslyn Compiler Deterministic Compilation
- How reproducible is a simple console app?
 - Demo script available in repository



Fennec Labs Reproduce

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Compare

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Future Idea - Fuzzing

#UCK26

- Fuzzing, or fuzz testing
 - Automated software testing method that uses a wide range of **invalid** and unexpected data as input to find flaws
- Good in C/C++ memory issues
- Can it be of any value with managed languages like .NET?



TIDALIS

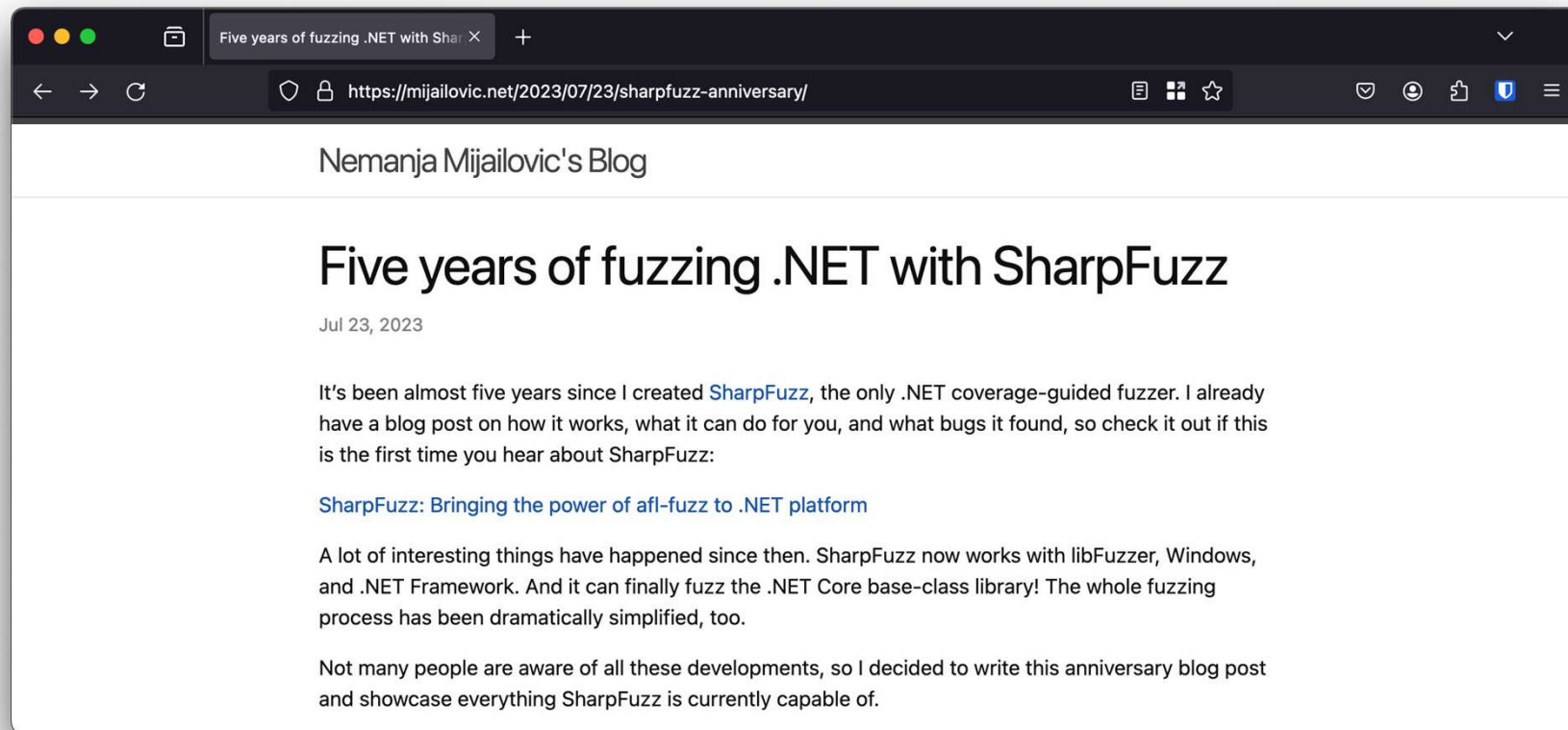


@niels.fennec.dev



@nielstanis@infosec.exchange

Fuzzing .NET & SharpFuzz



TIDALIS

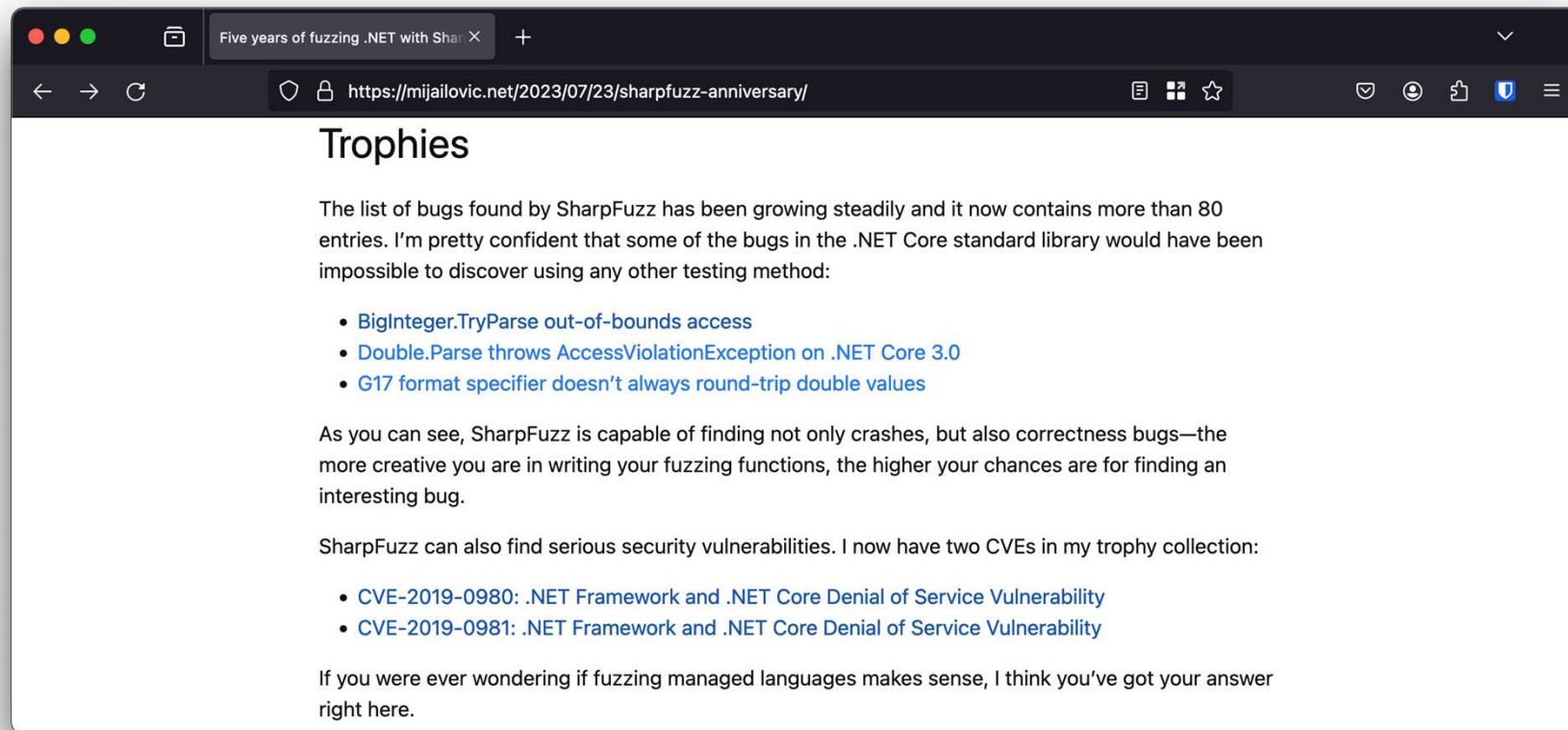


@niels.fennec.dev



@nielstanis@infosec.exchange

Fuzzing .NET & SharpFuzz

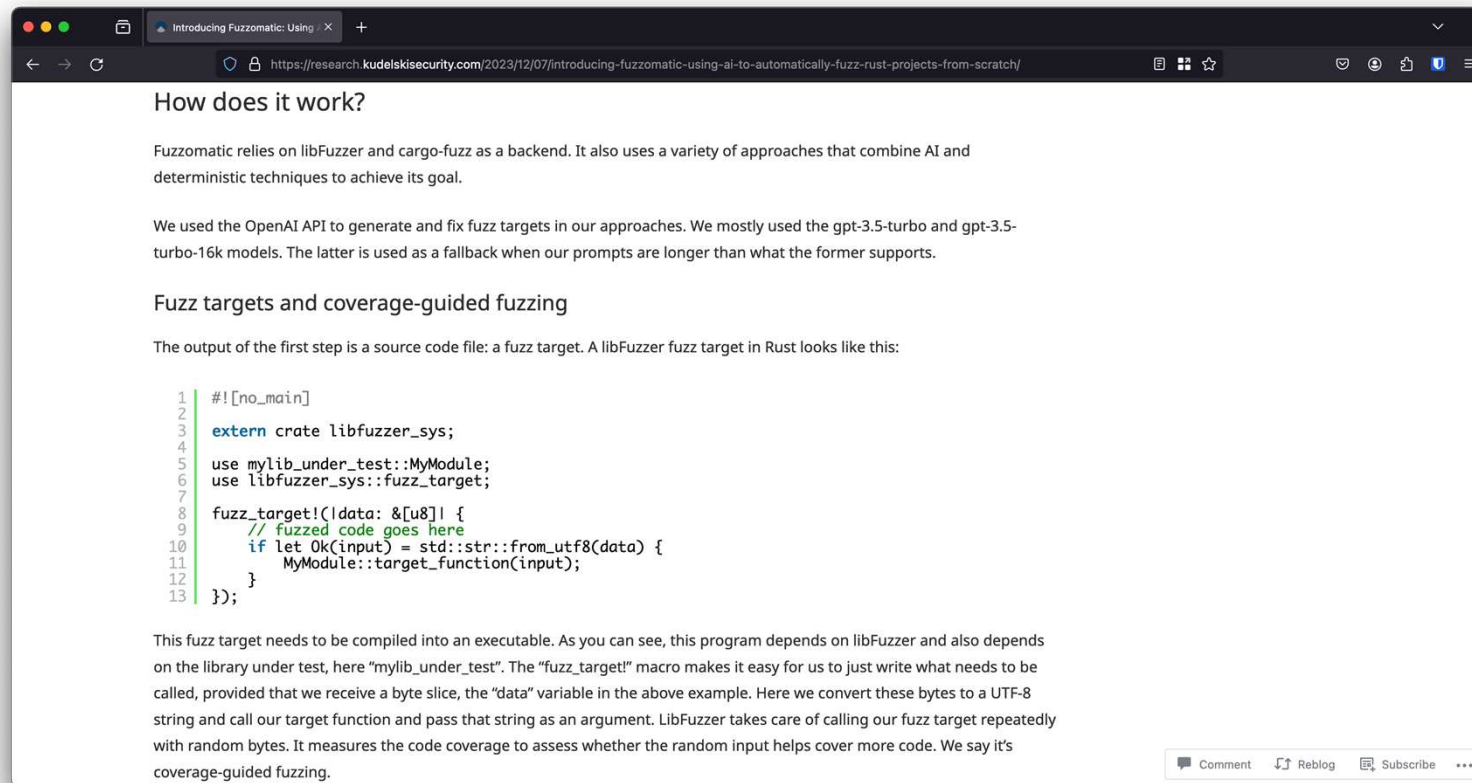


Fuzzing .NET – Jil JSON Serializer



```
public static void Main(string[] args)
{
    SharpFuzz.Fuzzer.OutOfProcess.Run(stream => {
        try
        {
            using (var reader = new
                System.IO.StreamReader(stream))
                JSON.DeserializeDynamic(reader);
        }
        catch (DeserializationException) { }
    });
}
```


Fuzzomatic: Using AI to Fuzz Rust



Future Idea – Dashboard & Feedback

#UCK26

- Local and remote dashboard?
- Ability to do review and share security related data of project
- MCP server with embedded prompt resources to help security review
- Roslyn analyzer that can help guide the developer



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Conclusion



- Doing software security is hard and Software Security Professionals don't scale!
- Package eco-systems have been wild west lately
- Goal is to have Fennec Labs help with reviewing and sharing security details for all audiences
- Allowing everyone to make better informed decisions and can act if needed.



Thank you!

#UCK26

- niels at fennect.dev
- <https://github.com/nielstanis/updateconfkrakow2026>
- Questions?



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange